The Power of Go - Tools

John Arundel

November 19, 2021

THE POWER OF GO TOOLS

ADVANCED SOFTWARE ENGINEERING IN GO

JOHN ARUNDEL

Contents

7. Filesystems **3**
Working with files . . . 3
Organising files . . . 4
A simple file counter . . . 4
Handling folders recursively . . . 5
Filesystems . . . 6
Matching files by name . . . 7
Walking the tree . . . 8
A file-counting tree-walker . . . 9
The `fs.FS` abstraction . . . 10
Any path-value map is a “filesystem” . . . 10
Testing memory-based filesystems . . . 10
Generalising the file counter . . . 11
Benchmarking slow operations . . . 12
Writing benchmark functions . . . 13
Using `fs.FS` in APIs . . . 14
Takeaways . . . 15
Going further . . . 16

7. Filesystems

To understand recursion, you must first understand recursion. —Traditional

In previous chapters we've read data from files, and written data to them. We haven't worried too much about exactly how those files are stored, treating them instead as simple abstractions with byte-oriented interfaces: `io.Reader` and `io.Writer`.

That's fine, but Go programmers are often called upon to write tools and utilities that manipulate files directly on disk (or the equivalent of disk, such as cloud storage).

Working with files

Because the exact semantics of working with files depends on your operating system, the Go standard library provides many useful facilities in the `os` package. For example, you can work directly with the `*os.File` type, which represents a disk file as seen by the operating system.

This is great when all you need to do is operate on some individual file, but we're often concerned with *collections* of files. Let's take a moment to talk about how these are stored and addressed, at least in Unix-like operating systems.

There are lots of different data storage technologies, including spinning magnetic disks, solid-state disks, RAM disks, and so on. As programmers, we almost never need to worry about any of these implementation details. One of the reasons Unix has been so successful is that it gives us some very simple abstractions to deal with files:

- We can *address* a file using a string called a *pathname*. This uniquely specifies some file and how to find it, relative to some *filesystem*.
- Having found the file we want, we can access its data using very simple *read*, *write*, and *seek* operations.

You're already familiar with reading and writing files, and *seeking* is the process of moving the current read/write position to some specific offset in the file. For example, if you have a 100GiB data file and you only want the last byte of it, you don't have to read all the preceding bytes. You can seek directly to one byte before the end, and then read it. In practice, we're not usually working at such a low level of abstraction in Go programs.

Organising files

On the other hand, we're often very concerned with how to address files: finding them, creating them, deleting them, renaming them, and so on. So how are files organised? The simplest imaginable way would be a *flat file system*: each file has a unique name, and that's it.

Flat file systems aren't very convenient, because of the uniqueness constraint, and there's often a fairly small limit on the length of filenames, which in turn limits the number of files we can have on a given disk or filesystem.

A much more flexible scheme is the *hierarchy*: grouping files into collections called *directories* or *folders*. Now filenames only have to be unique within their containing folder, and since folders can contain folders, this structure is recursive. Here's an example:

```
.
  notes.txt
  letters
      bank.txt
      archive
          bank.txt
```

There are several files here, some with the same name, but that's allowed because they're not in the same folder: each file still has a unique *path*. The files and folders form a *tree* structure. At the top level (the *root* of the tree), there's a file `notes.txt`, and a folder `letters`.

The `letters` folder in turn contains the file `bank.txt`, and another folder, `archive`. We say that `archive` is a *subfolder* of `letters`. This contains another file `bank.txt`, and so on.

We can describe this whole structure as a tree, but also refer to individual *subtrees* within it, such as the subtree rooted at `letters`. The files, folders, and relationships between them, are collectively called a *filesystem*. Usually, a filesystem corresponds to some physical disk or storage system.

It's pretty easy to implement folders, by the way: we can give certain files a special marker or piece of metadata that tells the operating system it's a folder, instead of a regular file. This special file can contain a list of the files (and folders) contained in the folder, each with the necessary information to tell the OS how to find the data associated with that file.

As a user of such a filesystem, how do we address individual files and folders? In Unix-like systems, pathnames are usually written by specifying each folder's name, in descending order from the root, separated by a slash (/). For example, these are the pathnames of the text files shown above:

```
./notes.txt
./letters/bank.txt
./letters/archive/bank.txt
```

A simple file counter

Suppose we have been tasked with writing a tool that will count the number of Go source files contained in some tree (for example, a project repository). How might we go about that using the operations available in

the os package?

The argument to the program will be the pathname of some folder. For example:

```
/Users/john/code/games/lander
```

The first thing the program would need to do would be to get the list of all files in this folder (remembering that subfolders are also a kind of file). To do this, we can use os.ReadDir:

```
files, err := os.ReadDir("/Users/john/code/games/lander")
```

Since this path might not exist or not be readable by us, we need to handle a potential error from ReadDir. Assuming there is none, we now have a files variable containing a slice of os.DirEntry values. Each of these contains metadata about a specific file.

We need to inspect the name of each file in turn, so we could range over this slice and use the handy path.Ext function to check if a file's extension is .go, and increment some counter value if the filename matches. Something like this:

```
func countGoFiles(folder string) (count int) {
    files, err := os.ReadDir(folder)
    if err != nil {
        return 0
    }
    for _, f := range files {
        if path.Ext(f.Name()) == ".go" {
            count++
        }
    }
    return count
}
```

Great! Are we done?

Handling folders recursively

Well, not quite: we only counted the Go files in the target folder. What if it contains subfolders with Go files in, or subfolders within subfolders? We'll need an extra step to detect if a given file is in fact a folder. We can call f.IsDir to ask if this is the case.

And what should we do if we find a folder? We could call os.ReadDir on it to get the list of *its* contents, but there's no limit to how deep this could go. Instead, we would do better to put our code in some function that we could call recursively, to match the recursive nature of the hierarchy.

If this function finds a folder, it can call itself to get the count of Go files within the tree rooted at that folder, and so on. Eventually, having recursed into all the folders in the tree, the final result will be the total number of Go files.

Suppose we had a tree like this, for example:

```
findgo
  file.go
  subfolder
     subfolder.go
  subfolder2
      another.go
      file.go
```

There are four files whose names end in .go, so when we run our file finder against this tree, we would expect the output "4".

Something like this might work:

```
func main() {
    fmt.Println(countGoFiles("findgo", 0))
}
```

```go
func countGoFiles(folder string, count int) int {
    files, err := os.ReadDir(folder)
    if err != nil {
        // skip
        return count
    }
    for _, f := range files {
        if f.IsDir() {
            count = countGoFiles(folder+"/"+f.Name(), count)
        }
        if path.Ext(f.Name()) == ".go" {
            count++
        }
    }
    return count
}
```

(Listing 7.1)

This is a little tricky to write, and even trickier to read. It's also not very efficient: each of those recursive function calls eats up a little bit of stack memory, so the deeper the hierarchy, the bigger the memory footprint of our program.

In practice it would take a fair bit more code to make this program really robust and flexible, never mind portable. And we'd hate to have to write all of this every time we wanted to do some operation on a file tree. Can't we do better?

Filesystems

Dealing with file trees of arbitrary depth is so common that the Go standard library provides an abstraction to make this easier: the *filesystem*. The `io/fs` package defines an interface `fs.FS` that represents a tree of files.

Let's refer to an `fs.FS` value as a "filesystem" from now on, keeping in mind that the term also has the more general meaning we discussed earlier in this chapter. How can we create a filesystem representing the tree of files rooted at a specific disk folder, then?

Opening a folder as an `fs.FS` is straightforward. We can do this by calling `os.DirFS`:

```go
fsys := os.DirFS("findgo")
```

Notice that there's no error here. That's because we haven't actually done any disk operations yet; we've just created the abstraction representing the file tree rooted at `findgo`. If there doesn't happen to be such a path, or we're not allowed to read it, well, too bad: we'll find that out when we try to do something that involves reading it.

An `fs.FS` by itself doesn't do much. You might be surprised by how small its method set is:

```go
type FS interface {
    Open(name string) (File, error)
}
```

(https://pkg.go.dev/io/fs#FS)

The only thing we can do by calling methods on the filesystem, then, is to `Open` some path within it. For example:

```go
f, err := fsys.Open("file.go")
```

If this path doesn't correspond to any existing file, then there will be some error (specifically, an `fs.PathError`). But if there isn't, then we successfully obtained some `f` of type `fs.File`. Great! So what's that?

```go
type File interface {
    Stat() (FileInfo, error)
    Read([]byte) (int, error)
    Close() error
}
```

(https://pkg.go.dev/io/fs#File)

Again, a very simple interface. All we can do with an `fs.File` is *stat* it (ask for its metadata, such as its name or permissions), or use it as an `io.ReadCloser`, just like a regular `*os.File`.

One interesting thing we can't do is *write* to it. Filesystems in Go are read-only, which is fine. A Go proverb says:

The bigger the interface, the weaker the abstraction.

In other words, if we're interested in operations on files *as part of a tree structure*, then `fs.FS` is the ideal abstraction for us: it's not cluttered up with stuff related to writing, modifying, moving, deleting, or changing permissions on files.

Once we know a file's pathname, we can always use the existing `os` machinery if we need to modify or write to it.

When you're dealing with trees of files, use `fs.FS` rather than trying to write the recursion code yourself.

Matching files by name

So what filesystem-level operations might we want to do? With our file-counting program in mind, one useful tool is `fs.Glob`, which takes a filesystem and a pathname containing wildcards (for example, `"*.go"`) and returns the names of all the matching files:

```
matches, err := fs.Glob(fsys, "*.go")
```

An error here would indicate that the pattern is invalid in some way, while `matches` is a slice of strings containing the pathnames of the matching files. The *glob* patterns are very similar to those understood by the Unix shell; for example, in the command `ls *.go`.

Great! Does that mean we can write our file-counting program as simply as this?

```
func main() {
  fsys := os.DirFS("findgo")
  matches, err := fs.Glob(fsys, "*.go")
  if err != nil {
    log.Fatal(err)
  }
  fmt.Println(len(matches))
}
```

(Listing 7.2)

Not quite. `fs.Glob` interprets a glob like `*.go` as referring to all the `.go` files in a specific folder (for example, the root folder of the filesystem). So if we used this glob to count Go files, we'd miss any files that aren't in the root folder:

```
go run main.go
```

```
1
```

Couldn't we use another `*` to represent any folder, so that `*/*.go` would find all Go files in all subfolders? No, because while that glob would match Go files in `subfolder` or `subfolder2`, for example, it *wouldn't* match files in the root folder, or in any subfolders of subfolders. The `*` wildcard can only match non-separator characters in a pathname. We can't write a recursive glob, in other words.

Some languages and tools use a *double star* wildcard for this: for example, `**/*.go` would match all Go files anywhere in the filesystem. Alas, `fs.Glob` and the other Go standard library functions that use wildcards don't support this syntax. We'll have to keep thinking.

Walking the tree

Since a filesystem is recursive in nature, the actual recursion operation is always the same. In principle, the standard library could do this for us, and all we'd need to supply is the specific code to execute for each file or folder we find.

The `fs.WalkDir` function does exactly this. It takes a filesystem and some starting path within it, and recursively *walks* the tree, visiting every file and folder. For each one it finds, it calls some function that you provide, passing it the pathname. For example:

```
fs.WalkDir(fsys, ".", myFunc)
```

With our example tree, `myFunc` would be called once for each of the following pathnames:

```
.
file.go
subfolder
subfolder/subfolder.go
subfolder2
subfolder2/another.go
subfolder2/file.go
```

The first path, `.`, is the root folder of the filesystem itself.

So what could `myFunc` do with these paths? Well, it could look at each path and increment a counter if it ends in `.go`. This sounds promising. Let's see if we can write `myFunc`.

The function we pass to `fs.WalkDir` must have a signature that matches the type `fs.WalkDirFunc`:

```
func(path string, d DirEntry, err error) error
```

(https://pkg.go.dev/io/fs#WalkDirFunc)

When this function gets called, the `path` parameter will be the pathname of the current file, while the `DirEntry` contains the file's metadata. We'll see what `err` is for in a moment.

First, let's write a suitable `WalkDirFunc` for our file counter. It just needs to check the path and increment its counter if it decides it's found a Go file:

```
func matchGo(p string, d fs.DirEntry, err error) error {
    if filepath.Ext(p) == ".go" {
        count++
    }
    return nil
}
```

We can now pass `matchGo` as the function argument to `fs.WalkDir`:

```
fs.WalkDir(fsys, ".", matchGo)
```

In practice, and to avoid making `count` a global variable, we wouldn't write some named function `matchGo` and then refer to it. We would pass it as an anonymous *function literal* instead, and this is very idiomatic in Go:

```
var count int
fsys := os.DirFS("findgo")
fs.WalkDir(fsys, ".", func(p string, d fs.DirEntry, err error) error {
    if filepath.Ext(p) == ".go" {
        count++
    }
```

```
    return nil
})
fmt.Println(count)
```

(Listing 7.3)

So what's that `error` parameter about? It lets you control what happens if there's an error reading some folder. For example, suppose we don't have read permission to get the contents of `subfolder`, so when `WalkDir` tries to open it there'll be an `fs.ErrPermission` error. In this case, `WalkDir` would call our function with `"./subfolder"` and the error, and the function could decide what to do about it.

The simplest thing it could do would be to just return the error immediately, in which case `WalkDir` would stop walking and return the same error itself. That's usually not what we want, though. We wouldn't want a single permission error to crash a 17-hour file scanning job a few minutes before the end, forcing us to start all over again.

Instead, we can return the special error value `fs.SkipDir`, meaning "skip this folder, but carry on walking". We don't handle errors in our example program, so it will always just skip any folders it can't read, which isn't completely unreasonable. You might need more sophisticated error handling in production programs, though.

A file-counting tree-walker

It looks like using a filesystem and `fs.WalkDir` will work for our file-counting program, so let's see how to turn it into some tested Go package ready for a million users who want to find and count Go files. We'll call it, unimaginatively, `findgo`.

As usual, let's first think about the `main` function we'd like to write, with absolutely minimal paperwork. Something like this would be nice:

```
func main() {
    fmt.Println(findgo.Files(os.Args[1]))
}
```

(Listing 7.4)

It wouldn't actually be that simple, in practice, since we'd need to check that `os.Args[1]` exists, report errors, and so on. But the CLI isn't the point of this example, and we've covered all that machinery in previous chapters, so let's take it as read for now, and see how `findgo.Files` would work.

It would need to take the pathname of some folder as its argument, and it would walk the tree rooted at that folder counting Go files, in the way that we've already done as a proof of concept. Let's write a test for that.

```
func TestFiles(t *testing.T) {
    t.Parallel()
    want := 4
    got := findgo.Files("testdata/findgo")
    if want != got {
        t.Errorf("want %d, got %d", want, got)
    }
}
```

(Listing 7.4)

We'll copy our example tree of files into `testdata/findgo` so the test has something to work on. So the test is saying that if we call `Files` with this path, in which there are four Go files, it should return the answer 4. Over to you to make this work.

GOAL: Implement `Files`.

Well, we've already more or less done it, haven't we? We can take the code from our `main.go` proof of concept and move it straight into the `findgo` package:

```
func Files(path string) (count int) {
    fsys := os.DirFS(path)
    fs.WalkDir(fsys, ".", func(p string, d fs.DirEntry, err error) error {
        if filepath.Ext(p) == ".go" {
            count++
```

```
        }
        return nil
    })
    return count
}
```

(Listing 7.4)

Excellent! The program works perfectly on our little test tree. But we can imagine that a program with more complicated logic *might* run into problems, especially in large and complicated filesystems. How could we test cases like that?

The `fs.FS` abstraction

One problem with tests that use files on disk is that disk access is, in general, extremely slow (at least, compared to accessing data in memory). Old-school spinning disks have a pretty high *latency*: it takes a long time to start sending data, because the read head has to physically travel to a specific place on the disk first.

Solid-state disks are faster, or at least have lower latency, but the data still has to get to the CPU over some relatively low-bandwidth link, such as a USB bus. The technology is always changing, but it's a reasonable rule of thumb that we can access data in memory an order of magnitude or two faster than data on disk.

This is the kind of speedup that every programmer dreams of, so can we achieve it here? In general, could we test filesystem operations on some in-memory data structure instead of having to access a disk?

As we saw earlier in this chapter, `fs.FS` is an interface. Specifically, an interface with the method `Open`. This effectively maps some slash-separated pathname to an `fs.File`, where `fs.File` is some file-like object that supports statting, reading, and closing. Such small interfaces make for powerful abstractions.

If we think a little more generally, we could treat a filesystem as a kind of key-value database. The keys here are pathnames, and each key is associated with a unique value representing a file. Specifically, this is a *path-value* database, because the keys obey a hierarchical pattern. In our example, the key `subfolder/subfolder.go` maps to whatever data is represented by the `subfolder.go` file.

Any path-value map is a "filesystem"

In fact, there are lots of things that fit this abstraction. The Web itself is a kind of path-value database, isn't it? A URL is a hierarchical, slash-separated pathname that maps to some specific data. A JSON, YAML, or CUE document has the same implicit structure. So does a Go import path, or the files contained in a ZIP archive.

In principle, we could use an `fs.FS` to access any of these things. As long as there's some concept of a hierarchical pathname mapped to a piece of data, the filesystem abstraction can be useful.

We could even imagine implementing `fs.FS` with some simple data type based on a Go map. All we'd need to do is give it an `Open` method, that maps string keys to some value that satisfies `fs.File`. Since `fs.File` is also a small interface, that's easy to do.

This would be very useful for testing any code that works with filesystems. While a tree of files on disk is a perfectly good `fs.FS`, it's a very slow one compared to a Go map, or anything else that lives in memory.

Instead, we could create simple pieces of test data, such as our Go file tree, by using a map literal. And the test would execute very fast, because it wouldn't need to make any disk accesses: everything would be in memory.

Testing memory-based filesystems

As it happens, we don't need to implement such a type ourselves, because the standard library already provides one: `fstest.MapFS`. This satisfies the `fs.FS` interface, and it's easy to construct `MapFS`-based filesystems in code.

Let's see how to rewrite our test for `Files` using a `MapFS` instead of regular disk files.

```
func TestFilesInMemory(t *testing.T) {
    t.Parallel()
    fsys := fstest.MapFS{
```

```go
        "file.go":                {},
        "subfolder/subfolder.go": {},
        "subfolder2/another.go":  {},
        "subfolder2/file.go":     {},
    }
    want := 4
    got := findgo.Files(fsys)
    if want != got {
        t.Errorf("want %d, got %d", want, got)
    }
}
```

(Listing 7.5)

A `MapFS` is some kind of Go map, as the name implies. Specifically, it's a map of `string` (the pathname) to `*MapFile` values.

In this test, we first set up our variable `fsys` as a `MapFS` literal where each of the keys represents one of our test files in `testdata/findgo`.

Notice that we haven't included the folders `subfolder` and `subfolder` as explicit keys in this map. We could, but it's not necessary, because they're implied by the pathnames of the files within them. For example, the path `subfolder2/file.go` implies a containing folder `subfolder2`. The `MapFS` implementation will "simulate" that folder for us if it needs to.

A `MapFile` is the "fake file" object that satisfies `fs.File`, and it has fields for storing data as a `[]byte`, file permissions, and so on. In other words, everything we're allowed to do with an `fs.File`, we can do with a `MapFile`.

As it happens, the code under test here only needs to look at the pathnames, so the `MapFile` values don't matter at all. We can leave them empty, so each element of this map is just the empty struct `{}`.

When you need to test some function that takes a filesystem, it's faster to give it a `MapFS` than a real tree of files on disk.

What should happen when we call `Files` with this filesystem? Well, just the same as before: it should recursively walk the filesystem looking for `.go` files and reporting the total count. The result should be 4, just as with the tree of real files on disk.

Indeed, `Files` doesn't know that it's *not* looking at disk files, and that's the point. We've generalised it to something that simply queries a path-value database.

We'll need to make a couple of minor changes to both `Files` and our previous `TestFiles` function to make this work. Can you see what to do?

GOAL: Get all tests passing.

Generalising the file counter

We need to update `Files` to take an `fs.FS` as its parameter instead of a pathname. And since we're *receiving* the filesystem now, we needn't open it ourselves using `os.DirFS`: we can remove that call.

Here's the modified `Files` function:

```go
func Files(fsys fs.FS) (count int) {
    fs.WalkDir(fsys, ".", func(p string, d fs.DirEntry, err error) error {
        if filepath.Ext(p) == ".go" {
            count++
        }
        return nil
```

```
    })
    return count
}
```

(Listing 7.5)

Next, we'll update the other test: the one that calls `Files` on a disk-based filesystem. Instead of just passing the root path, it will now need to call `os.DirFS` itself to create the filesystem:

```
func TestFilesOnDisk(t *testing.T) {
    t.Parallel()
    fsys := os.DirFS("testdata/findgo")
    want := 4
    got := findgo.Files(fsys)
    if want != got {
        t.Errorf("want %d, got %d", want, got)
    }
}
```

(Listing 7.5)

We don't really need `TestFilesOnDisk` anymore, since it just replicates what `TestFilesInMemory` does, only it's a lot slower.

But before we delete it, it would be interesting to know *how* much slower it is. Let's do a little science.

Benchmarking slow operations

Go provides excellent *benchmarking* facilities for measuring performance, in the same standard `testing` library we've been using. Before we see how to use them to write a benchmark for `Files`, let's think about the problem a little. What's actually involved in measuring the speed of some function?

If we didn't have built-in benchmarking facilities, and we wanted to measure the time elapsed during some potentially slow operation, what could we do?

One approach might be something like this:

```
start := time.Now()
slowOperation() // could take a while
elapsed := time.Since(start)
fmt.Println(elapsed)
```

By finding the current *wall-clock time* before and after the operation, and subtracting one from the other, we get the elapsed time. Fine. But how reliable is that figure?

Not all that reliable, in fact: your computer is doing a lot of other things at the same time as running this program, and any of these might cause the operation to be faster or slower.

In other words, the environment is *noisy*, causing *jitter* in our time measurements: if we repeat the same operation many times, we'll get results that differ by essentially random amounts. If the jitter is large compared to the total time we're measuring, that's a problem.

To avoid this, it's a good idea to run the operation some large number of times (say a million), and then divide the total by that figure to get the *mean* time per operation. Hopefully the noise will mostly cancel out, giving us a more accurate result. So we'll call the function inside a loop that executes many times.

But how many times? If the function is very fast, we should run it very many times to get a statistically useful answer. But if it's slow, running it many times would make the test take a long time. So the number of times we need to run it depends on how long it actually takes.

We could imagine determining the number of loop iterations by trial and error, first calling the function a small number of times to see how long that takes, optionally running it more times, and so on, until we've reached some overall time limit (ten seconds, let's say).

Writing benchmark functions

Once again, the standard library can take over this workload for us. All we need to do in a benchmark function is write a loop that calls our function N times, where N is some variable that can be set automatically by the test machinery. Here's what that looks like:

```
func BenchmarkFilesOnDisk(b *testing.B) {
    fsys := os.DirFS("testdata/findgo")
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        findgo.Files(fsys)
    }
}
```

(Listing 7.5)

Just as test function names begin with the word `Test`, benchmark function names begin with the word `Benchmark`. And just as tests take a `*testing.T` parameter that controls the test execution, benchmarks take an analogous `*testing.B` parameter that controls the benchmark execution. We'll see exactly how that works in a moment.

We first want to measure how long it takes `Files` to search a disk-based filesystem, so we create one using `os.DirFS` in the same way as the regular test.

The stopwatch is running from the moment the benchmark functions starts. But since we don't want the call to `os.DirFS` to be included in the benchmark timing, we then call `b.ResetTimer` to reset the clock to zero: "start timing from now!"

And now we write the loop we discussed earlier, to call our function a variable number of times. In fact, it will loop `b.N` times, where `b.N` is controlled by the benchmark machinery so as to make the overall time around ten seconds, to give a more reliable result.

To run this benchmark, use the command:

```
go test -bench .
```

```
goos: darwin
goarch: amd64
pkg: findgo
cpu: Intel(R) Core(TM) i7-3615QM CPU @ 2.30GHz
BenchmarkFilesOnDisk-8   8335    127055 ns/op
PASS
```

Because a benchmark is just a special kind of test, we're using the `go test` command, which will also run our regular tests. This makes sense, because there's no point benchmarking anything unless the tests are passing: a function can be arbitrarily fast if it doesn't have to be *correct*.

However, we also supply the `-bench .` flag, asking `go test` to also run any benchmarks whose names match the regular expression "`.`". Since the "dot" wildcard matches any character, this will run all benchmarks defined in the package.

The output gives us some contextual information about the system we're running the test on (a `darwin` kernel, signifying macOS, on the `amd64` architecture, specifically an Intel Core i7 clocked at 2.3GHz).

It then reports the results of the benchmark:

```
BenchmarkFilesOnDisk-8   8335    127055 ns/op
```

The `-8` following the benchmark name indicates that 8 CPU cores were available to Go, in case we need to know that: it might be important for concurrent programs.

The next number, `8335`, indicates how many times the loop executed overall, and thus how many times `Files` was called.

Finally, the figure `127055` tells us that the mean time per operation (that is, per call to `Files`) was a hundred and twenty-seven thousand nanoseconds and change. That is to say, about 127 microseconds.

Seems pretty fast! Can we do better using the `MapFS`-based filesystem? Let's find out:

```go
func BenchmarkFilesInMemory(b *testing.B) {
    fsys := fstest.MapFS{
        "file.go":                {},
        "subfolder/subfolder.go": {},
        "subfolder2/another.go":  {},
        "subfolder2/file.go":     {},
    }
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        findgo.Files(fsys)
    }
}
```

(Listing 7.5)

Here's the result:

```
go test -bench .
```

```
...
BenchmarkFilesOnDisk-8       8272  126715 ns/op
BenchmarkFilesInMemory-8   176679    6540 ns/op
```

Looking at the last column, we can see that the disk-based benchmark again took around 127 microseconds, while the memory-based one took only 6.5 microseconds. That's about twenty times faster, or about one order of magnitude, roughly what we expected.

To compare the performance of two implementations, use `Benchmark` functions.

So was it worth it? Well, we're all busy people, but most of us can probably spare a hundred microseconds to run a test. It's easy to imagine, though, that a 20x time saving could be significant on much larger test filesystems.

We also save the disk space, of course, and arguably it's clearer to see the file tree structure directly in the test code, rather than having to look at the `testdata` folder. So a `MapFS` is a neat tool to have available whenever we want to test code that operates on filesystems.

Using `fs.FS` in APIs

There's nothing stopping you from writing your own `fs.FS` implementation, and it's quite straightforward. Indeed, whenever you're writing Go code to deal with data that could in principle be addressed as a path-value tree, you might like to consider accepting an `fs.FS` as input, or making your data type satisfy `fs.FS` itself. It all helps to make your libraries more flexible, useful, powerful, and friendly.

We can see the effect of this with our file-counter example. Initially, because it took a disk pathname, the only thing we could use it to search was a disk-based filesystem. Now that we've updated it to accept `fs.FS`, it can operate on anything satisfying that interface. Our test can pass it a `MapFS` and it works just fine.

So what else would work? We mentioned earlier some examples of other things that satisfy `fs.FS`. Just for fun, let's try `Files` with a filesystem derived from a ZIP archive: after all, it should work, shouldn't it?

First, let's zip up our test `findgo` folder and its contents using the `zip` command. If you don't have that command, you can use anything that creates standard ZIP files, including the macOS Finder's "Compress" action.

```
cd testdata
```

```
zip -r findgo.zip findgo/
```

```
adding: findgo/ (stored 0%)
```

```
adding: findgo/subfolder/ (stored 0%)
adding: findgo/subfolder/subfolder.go (stored 0%)
adding: findgo/subfolder2/ (stored 0%)
adding: findgo/subfolder2/another.go (stored 0%)
adding: findgo/subfolder2/file.go (stored 0%)
adding: findgo/file.go (stored 0%)
```

All these files are empty, which is why zipping them doesn't seem to save much space, but that's not the point: we just want a ZIP file to play with. Now, how do we open it as a filesystem?

Helpfully, Go provides facilities for reading ZIP files in the standard library package `archive/zip`:

```go
func TestFilesInZIP(t *testing.T) {
    t.Parallel()
    fsys, err := zip.OpenReader("testdata/findgo.zip")
    if err != nil {
        t.Fatal(err)
    }
    want := 4
    got := findgo.Files(fsys)
    if want != got {
        t.Errorf("want %d, got %d", want, got)
    }
}
```

(Listing 7.5)

We call `zip.OpenReader` with the pathname of our test ZIP file, and the result is a value that satisfies `fs.FS`, so we can pass it directly to `Files`. And, of course, it gives us the correct answer:

```
PASS
```

Just by using the `fs.FS` abstraction instead of talking directly to the disk, we made our `Files` function work with ZIP archives, with no extra effort. And if you expose this type in your own APIs, other people will be able to use *your* `fs.FS` values in their own code.

If your function operates on trees of files, or sets of path-value pairs, consider taking an `fs.FS` parameter instead of a root path.

Takeaways

- Go's `os` package provides some powerful tools for working with individual files, but it can be awkward to use when trying to operate on lots of nested files and folders.
- Instead, use the filesystem interface `fs.FS` to abstract *trees* of files (or more generally, any collections of files addressed by hierarchical paths).
- Filesystems are read-only, but you can use them to *find* the files you want, then use the `os` package to make changes to them.
- Use `fs.WalkDir` to perform some operation for every file in a filesystem.
- When testing code that works with filesystems, use `fstest.MapFS`: it's much faster than accessing real disk files.
- If you need to simulate the behaviour of some unusual kind of filesystem for testing, implement `fs.FS` yourself.
- Naïve speed measurements, by comparing wall-clock times, are unreliable: use the `testing` library's benchmark facilities instead.

Going further

If you want some ideas for ways to get more practice with the ideas described in this chapter, here are some mini-project suggestions.

1. Write a Go package that finds all files in a given filesystem older than a specified duration (for example, 30 days). That is, files whose "last modified" date is 30 days or more before the present.

   Test your package using a `MapFS`. Check that it works on a disk filesystem.

2. Add a CLI that lets users find and delete files by age within a specified tree.

   For example, something like the following:

   ```
   prune -age 90d /backup
   ```

   would remove all files not modified in the last 90 days from anywhere in the tree rooted at `/backup`.

3. Add the ability to pass in a function to the package that determines if the old file is a candidate for deletion.

   For example, users should be able to find all files of the specified age for which some supplied "candidate for deletion" function returns `true`, indicating that they can be deleted. Test this using a simple example function.

   The package should be able to report the names of all the files that are candidates for deletion, as well as those that were ignored because too new, and separately, ignored because not candidates.

   A common use case for a tool like this is pruning old backups from servers, to avoid them filling up the disk. But we usually don't want to remove a backup file unless we can confirm it has been archived to cloud storage, or just somewhere off the machine, hence the "candidate for deletion" function to check this.

   If a particular backup has not been successfully archived for some reason, the tool needs to report that fact to the operator, or the file will simply sit there taking up space forever.